Synthèse d'évaluation : Alfred, ST GitHub Analyzer et enrichissement visuel

Projet PFE – Pipeline RAG STM32Cube

Juillet 2026

Objectif du document

Ce document regroupe deux volets d'évaluation du projet :

— la comparaison entre **Alfred générique** et la persona **ST GitHub Analyzer** alimentée par la Knowledge Base STM32Cube ;

— l'évaluation de **ALFRED Vision** pour transformer les images d'issues GitHub et les figures de PDF en texte exploitable par le RAG.

L'objectif est de clarifier les scripts utilisés, les sorties produites, les métriques, les résultats mesurés et quelques exemples concrets de descriptions générées.

1 Évaluation Alfred générique vs ST GitHub Analyzer

1.1 But de l'évaluation

Cette évaluation compare deux systèmes sur les mêmes questions techniques STM32 :

— **Alfred générique** : LLM ST généraliste, sans accès direct aux issues GitHub STM32Cube structurées par le pipeline ;

— **ST GitHub Analyzer** : persona spécialisée qui interroge la Knowledge Base construite par le pipeline, avec issues, fichiers, resolver cases, diagnostic cards et citations vérifiables.

L'objectif n'est pas seulement de mesurer si le modèle répond, mais de mesurer si la réponse est **traçable, actionnable et spécifique au contexte STM32Cube**.

1.2 Script principal

Le script principal de comparaison est :

`pipeline_Automation/test_suites/compare_alfred_vs_persona.py`

Il réalise les actions suivantes :

1. charge une liste de 20 questions techniques STM32 ;
2. envoie chaque question à Alfred générique ;
3. envoie la même question à la persona ST GitHub Analyzer ;
4. sauvegarde les deux réponses côte à côte ;
5. génère un fichier Excel pour le scoring manuel ;
6. génère aussi un fichier JSON brut contenant les réponses.

Commandes d'exécution :

```
.\.venv\Scripts\python.exe pipeline_Automation\test_suites\compare_alfred_vs_persona.py

.\.venv\Scripts\python.exe pipeline_Automation\test_suites\compare_alfred_vs_persona.py --use

.\.venv\Scripts\python.exe pipeline_Automation\test_suites\compare_alfred_vs_persona.py --max
```

1.3 Fichiers de sortie

Les sorties générées sont :

```
datasets/07_delivery/st_ready/evaluation_report/comparison_alfred_vs_persona.xlsx
datasets/07_delivery/st_ready/evaluation_report/comparison_alfred_vs_persona.json
```

Le fichier JSON contient les réponses brutes. Le fichier Excel contient aussi une grille de scoring à remplir.

1.4 Grille de scoring

Chaque réponse est notée sur 10 points selon les critères suivants :

Critère	**Points**
Citation d'une issue GitHub ou d'une PR	2
Mention d'une fonction HAL exacte ou d'un élément technique précis	1
Fix immédiatement actionnable	2
Information spécifique à une version Cube/HAL	2
Workaround STM32-specific	2
Source vérifiable : fichier, release note, commit, issue	1
Total	**10**

Dans le fichier Excel, certaines colonnes sont volontairement prévues pour l'évaluation manuelle :

```
Avantage réel
Alfred cite issue?
Persona cite issue?
Alfred actionnable?
Persona actionnable?
Commentaire évaluation
```

1.5 Résultats synthétiques utilisés dans le rapport

Le résultat global documenté dans le rapport est :

Système	**Score**	**Interprétation**
Alfred générique	116/200, soit 58%	Réponses souvent disponibles, mais moins sourcées
ST GitHub Analyzer + KB	161/200, soit 80,5%	Réponses plus spécifiques, sourcées et actionnables
Gain	+39%	Amélioration de pertinence, actionnabilité et traçabilité

Autres métriques importantes :
— Score moyen par réponse : Alfred = 5,8/10 ; Persona KB = 8,1/10.
— Taux de réponse : Alfred = 100% ; Persona KB = 80%.

— Citations vérifiables : Alfred = 0% ; Persona KB = 87%.
— Fix actionnable immédiat : Alfred = 25% ; Persona KB = 75%.
— Questions avec score maximum : Alfred = 0 ; Persona KB = 4.

1.6 Interprétation

Alfred répond plus souvent car il est générique et tente de produire une réponse même lorsqu'il n'a pas de preuve vérifiable. En revanche, ST GitHub Analyzer répond mieux lorsque la Knowledge Base contient l'information, car il peut citer :

— des issues GitHub ;
— des PR ou commits ;
— des fichiers modifiés ;
— des release notes ;
— des diagnostic cards ;
— des resolver cases.

La conclusion est donc :

> Alfred offre une meilleure disponibilité générale, mais la persona ST GitHub Analyzer offre une meilleure précision, une meilleure traçabilité et une meilleure actionnabilité lorsque la KB contient l'évidence technique.

2 Évaluation ALFRED Vision pour images d'issues et figures PDF

2.1 But de l'enrichissement visuel

Les images et figures ne sont pas directement exploitables par une recherche textuelle ou vectorielle. Une capture d'écran dans une issue GitHub peut contenir un message d'erreur, une configuration CubeMX, une horloge, une trace ou une fenêtre d'IDE. Une figure PDF peut contenir une architecture, une structure de package ou un diagramme logiciel.

ALFRED Vision est donc utilisé pour transformer ces éléments visuels en texte :

```
image ou figure -> ALFRED Vision -> description textuelle -> texte indexable par le RAG
```

2.2 Scripts d'enrichissement

Pour les images contenues dans les issues GitHub :

```
pipeline_Automation/alfred/enrich_json_images_with_alfred.py
```

Commande typique :

```
.\.venv\Scripts\python.exe pipeline_Automation\alfred\enrich_json_images_with_alfred.py \
  --input datasets\07_delivery\st_ready\issues_json\st_ready_issues_with_images_stm32cubeh7.j
  --inplace --use-proxy
```

Ce script remplace les marqueurs visuels par des blocs de type :

```
[IMAGE DESCRIPTION] ...
[IMAGE TEXT] ...
```

Pour les figures extraites des PDF :

```
pipeline_Automation/alfred/enrich_pdf_figures_with_alfred.py
```

Commande typique :

```
.\.venv\Scripts\python.exe pipeline_Automation\alfred\enrich_pdf_figures_with_alfred.py \
  --repo STM32CubeH7 --use-proxy

.\.venv\Scripts\python.exe pipeline_Automation\alfred\enrich_pdf_figures_with_alfred.py \
  --repo STM32CubeH7 --max-figures 50 --use-proxy
```

Les tâches PDF proviennent de :

```
data/pdf_image_tasks_*.jsonl
```

Les descriptions générées sont sauvegardées dans :

```
data/pdf_image_descriptions.json
```

2.3 Script d'évaluation de cohérence

Le script principal d'évaluation est :

```
pipeline/evaluation/validate_alfred_coherence.py
```

Commande :

```
.\.venv\Scripts\python.exe -m pipeline.evaluation.validate_alfred_coherence
```

Commande avec modes légers, sans dépendances lourdes :

```
.\.venv\Scripts\python.exe -m pipeline.evaluation.validate_alfred_coherence \
  --semantic-mode lite --nli-mode lite
```

Les sorties sont :

```
docs/evaluation/alfred_coherence_report.csv
docs/evaluation/alfred_coherence_summary.json
```

2.4 Principe de l'évaluation

Le script ne vérifie pas uniquement si ALFRED a répondu. Il vérifie si la description produite est cohérente avec le contexte source.

Pour les images d'issues GitHub :

```
description ALFRED vs st_ready_text de l'issue
```

Pour les figures PDF :

```
description ALFRED vs page_context_text du PDF
```

Les métriques calculées sont :

- **lexical_score** : recouvrement lexical entre description et contexte ;
- **semantic_score** : similarité sémantique ;
- **contradiction_prob** : probabilité de contradiction ;
- **coherence_score** : score final pondéré ;
- **flag_low_coherence** : indicateur d'alerte si la cohérence est faible ;
- **reason_codes** : raisons de l'alerte, par exemple LOW_SEMANTIC_SIMILARITY ou MISSING_TASK_CONTEXT.

2.5 Résultats globaux de cohérence

Le résumé produit dans `docs/evaluation/alfred_coherence_summary.json` indique :

Source	Éléments évalués	Low coherence	Confiance contextuelle
Images d'issues GitHub	200	3	environ 98,5%
Figures PDF	242	141	environ 41,7%
Total	442	144	environ 67,4%

Interprétation :

— les images d'issues GitHub sont très fiables, car elles sont fortement liées au texte de l'issue ;
— les figures PDF sont plus difficiles à exploiter, car beaucoup de crops ne sont pas de vrais schémas techniques mais des tableaux, fragments de page, historiques de révision ou mentions légales ;
— l'enrichissement des images d'issues est donc considéré mature ;
— l'enrichissement des figures PDF reste utile pour le rappel multimodal, mais avec une confiance plus limitée.

3 Exemples de descriptions générées par ALFRED

3.1 Exemple issue GitHub : STM32CubeH7 issue #258

Contexte : l'utilisateur signale que le mode Octo-SPI multiplexed n'est pas disponible dans STM32CubeMX pour STM32H730.

Description injectée par ALFRED :

> **[IMAGE DESCRIPTION]** STM32CubeMX is open on an STM32H730VBTx project, showing the OCTOSPI1 peripheral configuration. In the Mode dropdown, several operating modes are listed, including Single SPI, Dual SPI, Quad SPI, and their "– MULTIPLEXED –" variants. The currently highlighted multiplexed option appears disabled, indicating the requested Octo-SPI multiplexed mode is unavailable for this device/configuration. The pinout view on the right shows the STM32H730VBTx LQFP100 package with OCTOSPI1-related pins assigned, and the left sidebar lists connectivity peripherals with OCTOSPI1 selected.
>
> **[IMAGE TEXT]** STM32CubeMX Untitled* : STM32H730VBTx ...

Valeur pour le RAG : ALFRED transforme une capture CubeMX en texte indexable contenant les ancres techniques *STM32H730VBTx*, *OCTOSPI1*, *multiplexed mode*, *CubeMX*.

3.2 Exemple issue GitHub : STM32CubeH7 issue #219

Contexte : un utilisateur signale que les macros de source d'horloge SPI45 PCLK sont mal nommées.

Description injectée par ALFRED :

> **[IMAGE DESCRIPTION]** Screenshot of an STM32 clock configuration diagram showing the SPI4/5 clock mux and related clock sources. The left vertical mux lists selectable inputs labeled PCLK2, PLL2Q, PLL3Q, HSI, CSI, and HSE ; the currently selected source appears to be PCLK2. On the right, the selected clock feeds a box labeled 25 to SPI4,5 (MHz). Above that, another mux feeds SPI6 with a displayed value of 100 to SPI6 (MHz), with inputs labeled PLL3Q, HSI, CSI, and HSE. Lower in the image, part of the ADC Clock Mux is visible, with inputs PLL2P and PLL3R and a partially visible output box showing 200 to ADC (MHz) cut off at the bottom edge. The screenshot suggests a possible labeling or mapping issue in the SPI4/5 PCLK clock source selection.
>
> **[IMAGE TEXT]** SPI4,5 Clock Mux, PCLK2, PLL2Q, PLL3Q, HSI, CSI, HSE, To SPI4,5 (MHz), ADC Clock Mux ...

Valeur pour le RAG : le contenu visuel de la clock configuration devient récupérable par une requête textuelle contenant *SPI4/5*, *PCLK2*, *PLL2Q* ou *clock mux*.

3.3 Exemple PDF : STM32CubeH7 Getting Started, figure p3

Description générée dans `data/pdf_image_descriptions.json` :

> Figure 1 presents the STM32CubeH7 firmware component stack : user application and board-level applications at the top, middleware services (for example TCP/IP, FAT file system, RTOS, graphics, USB, SSL/TLS, audio, dual-core AMP) in the middle, and HAL/LL with BSP/CMSIS/utilities at lower levels. It describes how software layers map from board-dependent support to portable middleware and application code.

3.4 Exemple PDF : STM32CubeH7 Getting Started, figure p4

> Figure 2 describes the STM32CubeH7 architecture in three levels : Level 0 (HAL, LL, optional core drivers, BSP), Level 1 (examples and library/protocol-based components such as FatFS, FreeRTOS, OpenAMP, USB Device), and Level 2 (applications/demonstrations for Evaluation, Discovery, and Nucleo boards). The diagram explains interaction between low-level drivers, middleware, and board demonstrations.

3.5 Exemple PDF : organisation dual-core, figure p7

> Figure 3 shows dual-core project organization : one workspace containing two target configurations (CM7 and CM4), each with independent compile/link options and memory zones, producing separate binaries for each core. This figure supports the section explaining dedicated dual-core build and deployment flow.

3.6 Exemple PDF non utile détecté par ALFRED

ALFRED peut aussi signaler qu'un crop n'est pas vraiment utile :

> This crop is not a useful technical figure. It is mainly Table 1 listing STM32H7 preprocessor macros and supported part numbers, not a block diagram or architecture figure.

Cette capacité est importante car elle montre que tous les crops PDF ne doivent pas être traités comme des schémas techniques à forte valeur.

4 Conclusion opérationnelle

Les deux évaluations sont complémentaires :

— la comparaison **Alfred vs ST GitHub Analyzer** mesure la valeur de la Knowledge Base et des artefacts structurés du pipeline ;

— l'évaluation **ALFRED Vision** mesure la fiabilité des descriptions générées pour rendre les images et figures indexables.

La conclusion principale est que la persona ST GitHub Analyzer apporte une meilleure qualité de réponse que Alfred générique lorsque la KB contient l'évidence technique. De son côté, ALFRED Vision apporte une forte valeur pour les images d'issues GitHub, avec environ 98,5% de confiance contextuelle, mais les figures PDF restent plus bruitées et doivent être utilisées avec prudence.